

# Manual do Usuário — Segmentação Semântica de Imagens Agrícolas usando NDVI e NDWI

Este manual ensina, do zero, como instalar, configurar e usar o projeto: por onde começar, como preparar os dados, treinar o modelo, acompanhar o treino e avaliar os resultados. O objetivo é garantir **reprodutibilidade**: seguindo estes passos, outra pessoa consegue rodar o pipeline completo e chegar aos mesmos resultados.

**Projeto da disciplina IA901 (Unicamp, 2026/1).** Segmentação semântica de anomalias em imagens aéreas agrícolas (dataset *Agriculture-Vision*), avaliando se a adição dos índices espectrais **NDVI** e **NDWI** aos canais RGB melhora a segmentação. Para a fundamentação teórica, metodologia e referências, veja o README.md.

---

## Sumário

1. Visão geral em 1 minuto
  2. Estrutura do repositório
  3. Pré-requisitos
  4. Instalação passo a passo
  5. Configuração (`config.yaml`)
  6. Por onde começar — fluxos de uso
  7. Treinamento
  8. Acompanhamento com TensorBoard
  9. Avaliação e visualização das predições
  10. Artefatos gerados
  11. Glossário
- 

## 1. Visão geral em 1 minuto

O projeto é um pipeline de **segmentação semântica multi-rótulo** com três etapas:

1. **Dados** — carrega o *Agriculture-Vision*, monta os canais de entrada (RGB, NIR e os índices NDVI/NDWI calculados na hora) e separa treino/validação/teste.
2. **Treino** — uma rede **FPN com backbone ResNet-50** (o encoder ResNet é pré-treinado por `weights='DEFAULT'` com seu último bloco sendo dilatado) é treinada em DDP (multi-GPU) para prever, por pixel, quais anomalias estão presentes.
3. **Avaliação** — calcula a **mIoU modificada** do *Agriculture-Vision* (métrica que aceita um mesmo pixel receber múltiplas classes).

O experimento central é trocar os canais de entrada (`rgb`, `rgbn`, `rgbnvw`) e comparar a mIoU, medindo o impacto dos canais NIR (n), NDVI (v) + NDWI (w).

---

| Componente                                            | Arquivo                       |
|-------------------------------------------------------|-------------------------------|
| Configuração central (lê o <code>config.yaml</code> ) | <code>src/constants.py</code> |
| Dataset e <i>DataModule</i>                           | <code>src/data.py</code>      |
| Modelo (FPN + ResNet-50)                              | <code>src/model.py</code>     |

---

---

| Componente                                                    | Arquivo                     |
|---------------------------------------------------------------|-----------------------------|
| Métrica (mIoU modificada)                                     | src/metrics.py              |
| Utilidades de treino (loss, scheduler, checkpoint, avaliação) | src/train_utils.py          |
| <b>Script de treino</b> (ponto de entrada)                    | src/main.py                 |
| Visualização e avaliação pós-treino                           | src/visual_utils.py         |
| Download do dataset                                           | src/scripts/load_dataset.sh |

---

## 2. Estrutura do repositório

Multispectral Agrícola/

```

config.yaml          # ÚNICO arquivo que você edita para configurar tudo
pyproject.toml       # dependências do projeto (gerenciadas pelo uv)
uv.lock              # versões travadas das dependências
README.md            # fundamentação teórica, metodologia, referências
Manual_do_Usuario.md # este manual

src/                  # código-fonte (pacote Python "src")
  constants.py        # lê o config.yaml e define caminhos/constantes globais
  data.py             # AgricultureVisionDataModule + AgricultureVisionDataset
  model.py            # FPN_ResNet50_Segmentation
  metrics.py          # ModifiedMIoU
  train_utils.py      # loss ponderada, poly-LR, checkpoint, loop de avaliação
  main.py             # script de TREINAMENTO (rodado via torchrun)
  utils.py            # visualização de amostras/predições + evaluate_model/test_model
  scripts/
    load_dataset.sh   # baixa e descompacta o dataset a partir do S3

notebooks/
  exploratory-data-analysis_eda.ipynb # análise exploratória (EDA)
  model-dev.ipynb                    # inspeção visual + mIoU no teste
  NDVI and NDWI generation/
    nvdi_ndwi.ipynb                  # geração standalone das imagens NDVI/NDWI

data/
  dataset/          # (criado pelo load_dataset.sh) - o dataset fica aqui
  cache/            # pos_weight.json e métricas em cache
  Datasheet__Agriculture_Vision.pdf

experiments/        # (criado durante o treino - ignorado pelo git)
  checkpoints/<canais>_v3/ # best.pth, latest.pth, final.pth
  tensorboard/<canais>_v3/ # logs do TensorBoard (train/ e val/)

assets/             # imagens usadas no README

```

---

### 3. Pré-requisitos

#### 3.1. Software

- **Sistema operacional:** Linux (Ubuntu 22.04+)
- **Python 3.11** (definido em `.python-version`).
- **uv** — gerenciador de ambiente/dependências (recomendado).
- **AWS CLI** — apenas para baixar o dataset com o script automatizado.
- **Git**.

#### 3.2. Hardware

- **Treinamento:** exige pelo menos uma GPU NVIDIA com CUDA. O script de treino usa DDP + backend nccl. O experimento original usou 2× TITAN Xp (12 GB), i7-7700, 32 GB RAM.
  - **Disco:** ~21 GB — dataset
- 

### 4. Instalação passo-a-passo

#### Passo 1 — Clonar o repositório

```
git clone https://github.com/luisso2/IA901-2026S1.git
cd "IA901-2026S1/projetos/Multiespectral Agrícola"
```

#### Passo 2 — Criar o ambiente e instalar as dependências

O projeto usa **uv**. Se ainda não tiver:

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

Depois, na raiz do projeto, crie o ambiente e instale tudo (PyTorch já vem da index `cu118`, configurada no `pyproject.toml`):

```
uv sync
```

Isso cria a pasta `.venv/`. Para rodar qualquer comando dentro do ambiente, use o prefixo `uv run` (ex.: `uv run python ...`) ou ative o ambiente com `source .venv/bin/activate`.

#### Passo 3 — Baixar o dataset

O dataset **Agriculture-Vision** é baixado de um bucket público do S3. O script `src/scripts/load_dataset.sh` cuida de tudo: verifica se já existe, baixa, descompacta os `.tar.gz` e apaga os compactados ao final.

Pré-requisito: ter o **AWS CLI** instalado (`pip install awscli` ou `sudo apt-get install -y awscli`).

```
chmod +x src/scripts/load_dataset.sh
./src/scripts/load_dataset.sh
```

O script baixa para `data/dataset/` e descompacta em `data/dataset/supervised/`. Ele pede confirmação antes de descompactar (mostrando o espaço necessário). Ao final, você terá a estrutura esperada do dataset:

```
data/dataset/supervised/Agriculture-Vision-2021/
  train/
    images/{rgb,nir}/      masks/      boundaries/      labels/<classe>/
  val/    (mesma estrutura, com labels)
  test/   (mesma estrutura, sem labels)
```

A pasta test/ original **não tem rótulos**. Por isso o projeto, por padrão, divide a pasta val/ em validação + teste (veja isSplitValidationSet na próxima seção).

#### Passo 4 — Apontar o config.yaml para o dataset

Edite o config.yaml e ajuste dataset\_path para o caminho **absoluto** da pasta que contém train/, val/ e test/. Veja a próxima seção para os detalhes.

---

### 5. Configuração (config.yaml)

Todo o comportamento do pipeline é controlado pelo config.yaml na raiz do projeto. É o **único arquivo que você precisa editar** para uso normal. Ele é lido por src/constants.py, que expõe as constantes para o resto do código.

```
dataset_path: /caminho/absoluto/para/Agriculture-Vision-2021

input_channels: 'rgbnvw'          # 'rgb', 'rgbn' ou 'rgbnvw'

classes_to_evaluate:
  - double_plant
  - drydown
  - endrow
  - nutrient_deficiency
  - planter_skip
  - water
  - waterway
  - weed_cluster
  # - storm_damage                # excluída: ausente no split de teste

code_variables:
  isSplitValidationSet: true       # divide o 'val' original em val + test
  taxForValidationSet: 0.5        # fração das farmlands que vai para validação
  seed: 7

model_hyperparameters:
  name: resnet50
  pretrained: true
  batch_size: 20                  # usado pelos NOTEBOOKS (não pelo main.py)
  is_weighted_loss: false        # liga/desliga a loss ponderada por classe
```

#### Referência dos campos

| Campo                             | O que faz                                                                                                                                                                                                                              |
|-----------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>dataset_path</code>         | Caminho absoluto da raiz do dataset (a pasta com <code>train/</code> , <code>val/</code> , <code>test/</code> ).                                                                                                                       |
| <code>input_channels</code>       | Quais bandas alimentam o modelo, <b>nessa ordem</b> . <code>r,g,b</code> = RGB; <code>n</code> = NIR; <code>v</code> = NDVI; <code>w</code> = NDWI. NDVI/NDWI são calculados na hora a partir de RGB+NIR (veja <code>data.py</code> ). |
| <code>classes_to_evaluate</code>  | Classes de anomalia a segmentar. A ordem define os canais de saída do modelo. <code>storm_damage</code> fica de fora por não existir no teste.                                                                                         |
| <code>isSplitValidationSet</code> | Se <code>true</code> , ignora a pasta <code>test/</code> (sem rótulos) e divide o <code>val/</code> por <i>farmland</i> em validação + teste. Se <code>false</code> , usa as pastas <code>val/</code> e <code>test/</code> como vêm.   |
| <code>taxForValidationSet</code>  | Fração das farmlands do <code>val/</code> que vira validação (resto vira teste). Só vale quando <code>isSplitValidationSet: true</code> .                                                                                              |
| <code>seed</code>                 | Semente para o embaralhamento do split (reprodutibilidade).                                                                                                                                                                            |
| <code>batch_size</code>           | <b>Usado apenas pelos notebooks.</b> O treino ( <code>main.py</code> ) usa um batch fixo no código.                                                                                                                                    |
| <code>is_weighted_loss</code>     | Se <code>true</code> , pondera a BCE por classe ( <code>pos_weight</code> ) para combater o desbalanceamento.                                                                                                                          |

**Atenção ao formato do `dataset_path`.** Use um **caminho simples**, como no exemplo acima. O código lê o valor como string literal (`yaml.safe_load`); ele **não** interpreta sintaxe de shell como `${DATASET_PATH:-/...}`. Se o seu `config.yaml` estiver com algo nesse formato, troque por um caminho absoluto direto, senão o carregamento do dataset falha.

Trocar `input_channels` muda também a pasta de saída dos checkpoints e logs (`experiments/checkpoints/<input_channels>/`), permitindo manter experimentos de configurações diferentes lado a lado.

## 6. POR ONDE COMEÇAR — fluxos de uso

Há dois caminhos típicos, dependendo do seu objetivo:

**A) Só quero ver os resultados / avaliar um modelo já treinado** → vá direto para a seção 9 (baixe os checkpoints prontos e abra o `model-dev.ipynb`).

**B) Quero rodar o pipeline completo do zero** → siga nesta ordem:

1. **Instalação** (seções 4 e 5) — ambiente + dataset + `config.yaml`.
2. **EDA** — abra `notebooks/exploratory-data-analysis_eda.ipynb` para entender o dataset, a distribuição de classes e validar que o split está funcionando.

3. (*Opcional/informativo*) **Geração de NDVI/NDWI** — notebooks/NDVI and NDWI generation/nvdi\_ndwi.ipynb gera e visualiza imagens dos índices. **Não é necessário para treinar:** o treino calcula NDVI/NDWI internamente. Esse notebook serve para inspeção visual dos índices.
4. **Treinamento** (seção 7).
5. **Acompanhamento** com TensorBoard (seção 8), em paralelo ao treino.
6. **Avaliação** e visualização (seção 9).

**Rodando os notebooks:** use o kernel do ambiente `.venv` (o `ipykernel` já é instalado pelo `uv sync`). Os notebooks importam `from src... import ...` e montam caminhos relativos como `../experiments/...`, então rode-os a partir da pasta `notebooks/`.

---

## 7. Treinamento

O treino é feito por `src/main.py`, que **só roda via `torchrun`** (ele espera as variáveis de ambiente do DDP, como `LOCAL_RANK`).

**Com 2 GPUs** (configuração original):

```
uv run torchrun --nproc_per_node=2 --standalone src/main.py
```

**Com 1 GPU:**

```
uv run torchrun --nproc_per_node=1 --standalone src/main.py
```

### O que acontece durante o treino

- Monta o modelo conforme `input_channels` do `config.yaml`, com a `conv1` da ResNet adaptada: R/G/B reaproveitam os pesos da ImageNet, NIR copia o peso do canal vermelho, e NDVI/NDWI começam zerados (a rede aprende do zero). Detalhes em `src/model.py`.
- **Loss:** BCE multi-rótulo mascarada pela ROI (`mask × boundary`) — só pixels válidos contam. Com `is_weighted_loss: true`, aplica `pos_weight` por classe.
- **Otimizador:** SGD (momentum 0.9 (não menciona no paper, acreditamos que eles devam ter utilizado), weight decay  $5e-4$ ) com *learning rate* poly (warmup → patamar → decaimento polinomial).
- **Avaliação periódica:** a cada 2500 iterações calcula a mIoU de treino e validação.
- **Checkpoints** (em `experiments/checkpoints/<input_channels>`):
  - `best.pth` — melhor mIoU de validação até o momento (grava-se somente os pesos);
  - `latest.pth` — estado completo (modelo + otimizador + scheduler + iteração) para **retomar o treino**;
  - `final.pth` — pesos ao final do treino.
- **Resume automático:** se existir `latest.pth`, o treino continua de onde parou.

**Hiperparâmetros fixos no código** (não vêm do `config.yaml`): em `src/main.py` estão definidos `total_iters = 25000`, `batch_size_por_gpu = 10` e `passos_acumulacao = 2` (batch efetivo =  $10 \times n^{\circ} \text{ de GPUs} \times 2$ ). Para mudá-los, edite o `main.py`. (O `batch_size` do `config.yaml` é usado só pelos notebooks.)

## 8. Acompanhamento com TensorBoard

Durante (ou depois) do treino, visualize loss, learning rate, mIoU e IoU por classe:

```
uv run tensorboard --logdir experiments/tensorboard --port 6008
```

Abra no navegador: <http://localhost:6008/>

Os logs ficam separados em `train/` e `val/` dentro de `experiments/tensorboard/<input_channels>/`.

---

## 9. Avaliação e visualização das predições

Use o notebook `notebooks/model-dev.ipynb`. Ele faz duas coisas:

- **Avaliação qualitativa** — `test_model(...)` desenha a predição sobre o RGB (ground truth vs. predição + mapa de erros) para uma amostra específica ou aleatória do dataset de **teste**.
- **Avaliação quantitativa** — `evaluate_model(...)` varre o split de teste inteiro e calcula a **mIoU modificada** + o IoU por classe, opcionalmente salvando em JSON.

### Usando checkpoints prontos (sem treinar)

Os checkpoints (`best.pth`, ~153 MB cada) estão disponíveis no Google Drive. Baixe a pasta `/checkpoints` e coloque em `experiments/` no projeto.

**Atenção ao caminho dos pesos.** O notebook monta o caminho como `../experiments/checkpoints/{INPUT_CHANNELS}/best.pth`, **enquanto o treino salva em** `experiments/checkpoints/{INPUT_CHANNELS}/`. Se você acabou de treinar, garanta que a pasta usada pelo notebook bata com a pasta gerada — renomeie a pasta ou ajuste a string do caminho no notebook (o trecho `f"../experiments/checkpoints/{INPUT_CHANNELS}/best.pth"`). O `INPUT_CHANNELS` (ex.: `rgbnvw`) precisa ser o mesmo com que o checkpoint foi treinado.

---

## 10. Artefatos gerados

| Artefato                                                                     | Onde                                                 | Gerado por                                                  |
|------------------------------------------------------------------------------|------------------------------------------------------|-------------------------------------------------------------|
| Checkpoints ( <code>best</code> , <code>latest</code> , <code>final</code> ) | <code>experiments/checkpoints/&lt;canais&gt;</code>  | <code>main.py</code>                                        |
| Logs do TensorBoard                                                          | <code>experiments/tensorboard/&lt;canais&gt;/</code> |                                                             |
| <code>pos_weight.json</code> (pesos da loss)                                 | <code>data/cache/</code>                             | treino, quando <code>is_weighted_loss: true</code>          |
| Métricas da EDA em cache                                                     | <code>data/cache/metricas_*.pkl</code>               | notebook de EDA                                             |
| JSON da mIoU de teste                                                        | onde você indicar em <code>save_path</code>          | <code>evaluate_model</code> no <code>model-dev.ipynb</code> |

---

## 11. Glossário

- **NDVI** (*Normalized Difference Vegetation Index*) =  $(\text{NIR} - \text{RED}) / (\text{NIR} + \text{RED})$ . Mede vigor da vegetação; valores altos = vegetação densa/saudável. Intervalo  $[-1, 1]$ .
- **NDWI** (*Normalized Difference Water Index*) =  $(\text{GREEN} - \text{NIR}) / (\text{GREEN} + \text{NIR})$ . Destaca água e umidade. Intervalo  $[-1, 1]$ .
- **NIR** — banda do infravermelho próximo; muito refletida por vegetação saudável e absorvida pela água.
- **ROI** (*Region of Interest*) — `mask × boundary`; só os pixels válidos dentro dela entram no cálculo de loss e métrica.
- **mIoU modificada** — versão da *mean Intersection over Union* do *Agriculture-Vision* que acomoda pixels com **mais de um rótulo** simultâneo (ver `src/metrics.py`).
- **DDP** (*Distributed Data Parallel*) — modo do PyTorch para treinar em várias GPUs em paralelo, orquestrado pelo `torchrun`.
- **FPN** (*Feature Pyramid Network*) — arquitetura do decoder que funde features em várias resoluções para segmentação (ver `src/model.py`).
- **farmland** — identificador da lavoura de origem de cada imagem (prefixo do nome do arquivo); o split val/test é feito por farmland para evitar vazamento entre conjuntos.